

Plateforme RandoVanGo

PLANIFICATION DE VOYAGES RANDONNEES VAN ITINERANTS



Nathalie BEDIEE
Développeur IA - ISEN Brest
Novembre 2025

Bloc de compétences 1 :
Réaliser la collecte, le stockage et
la mise à disposition des données

Table des matières

1.	PRÉSENTATION DU PROJET	2
1.1.	Genèse du projet.....	2
1.2.	Objectifs et spécifications techniques.....	2
2.	AUTOMATISATION EXTRACTION DES DONNÉES	3
2.1.	Architecture du pipeline ETL	3
2.2.	Extraction des sources	3
3.	AGRÉGATION ET NETTOYAGE	4
3.1.	Agrégation OSM	4
3.2.	Agrégation Park4Night	4
3.3.	Agrégation GPX	4
4.	BASE DE DONNÉES	4
4.1.	Base de données MySQL	4
4.2.	Base de données SQLite	5
4.3.	Base de données Mongo.....	5
4.4.	Requêtes SQL et optimisations	5
5.	FASTAPI (API REST)	5
6.	CONCLUSION	6
7.	ANNEXES.....	7
7.1.	Architecture technique global	7
7.2.	MCD BDD MySQL, données structurées nécessaire à la planification	7
7.3.	Schéma de la base de données MySQL	8
7.4.	MCD de la BDD SQLITE, authentification	8
7.5.	Diagramme de flux ETL Pipeline	9
7.6.	Diagramme de flux ETL des données Wikidata	9
7.7.	Diagramme de flux ETL des fichiers GPX	10
7.8.	Diagramme de flux ETL des données Park For Night.....	11
7.9.	Diagramme de flux ETL des données Open Street Map	12
7.10.	Diagramme de flux ETL de gestion des données météo	13
7.11.	Exemples de requêtes	13
7.12.	Diagramme de séquence authentification	14
7.13.	Documentation Swagger UI	14
7.14.	Gestion Planning Rando	15
7.15.	Arborescence projet.....	16
7.16.	Exemples de logs	17
7.17.	Vues du front	18

1. PRÉSENTATION DU PROJET

1.1. Genèse du projet

Le projet RandoVanGo est né d'une frustration : l'impossibilité d'organiser un séjour de randonnée en van sans jongler constamment entre cinq à six applications différentes (Météo France, Park4Night, MaRando, etc.). L'outil a été conçu pour centraliser ces informations vitales et résoudre cette dispersion des outils. Il fonctionne comme un planificateur de randonnées ([@ Gestion Planning Rando](#)) permettant à l'utilisateur, soit d'importer une trace GPX personnelle et de l'enrichir, soit de construire son voyage à partir des parcours intégrés à l'application. RandoVanGo offre ainsi une solution unique et complète aux randonneurs vanlifers.

Acteurs : Utilisateurs finaux (randonneurs, vanlifers), contributeurs qui peuvent fournir des nouveaux tracés de randonnées, des points d'intérêt (restaurants, commerces, points d'eau, stations-services, ...) ou des spots (lieu de stationnement et de nuitées)

1.2. Objectifs et spécifications techniques

Le projet s'articule autour d'un pipeline ETL (Extract-Transform-Load) automatisé, dont [@ Architecture technique global](#) intègre plusieurs sources de données :

- **APIs REST publiques** : Données météo (Open-Meteo), Points d'intérêt (POI) issus d'OpenStreetMap (via Overpass API), et données de patrimoine culturel (Wikidata via requête SPARQL).
- **Web scraping** : Spots pour des nuitées en van (Park4Night) en utilisant Selenium/Chromium.
- **Fichiers de données** : Traces de randonnées au format GPX (XML géospatial).

Les données extraites sont transformées en appliquant des règles métier (nettoyage, normalisation, agrégation). Elles sont finalement Chargées dans deux bases de données complémentaires :

- **MySQL** pour les données relationnelles structurées (villes, randonnées, spots, POI, météo).
- **MongoDB** pour les documents JSON géospatiaux (traces GPX complètes)

L'exposition des données est assurée par une API REST (FastAPI) documentée selon le standard OpenAPI. Cette API est consommée par un frontend développé en Flask.

Enfin, une base de données **SQLite** gère l'authentification et les rôles des utilisateurs (administrateurs et contributeurs) pour sécuriser l'accès et les actions spécifiques (comme l'upload de fichiers GPX).

Stack Technique :

- Backend & API : Python, FastAPI, Uvicorn
- Authentification : JWT, Passlib (Bcrypt)
- Frontend : Flask, Jinja2, HTML/CSS/JS, Leaflet.js (cartographie)
- Bases de données: MySQL (relationnelle via mysql-connector), MongoDB (NoSQL via PyMongo), SQLite
- ETL & Traitement : pandas, selenium/chromium, gpxpy (gestion GPX), geopy (Géocodage), requests (appels HTTP), threading (exécution parallèle), logging (suivi), Nominatim (OSM Geocoding)
- Conteneurisation : Docker, Docker Compose

Contraintes : Conformité RGPD (hachage bcrypt, traçabilité auth_logs) [@ Diagramme de séquence d'authentification](#)

Budget :Projet sans budget, utilisant que des outils open-source et APIs gratuites.

Méthodologie : Développement itératif avec cycles courts

- Versionnage Git avec branches : [@ GitHub/natbediee/randovango](#)
- Tests manuels via Swagger UI (FastAPI)
- [@ Arborescence structurée](#)
- logs structurés (backend/logs/)

2. AUTOMATISATION EXTRACTION DES DONNÉES

2.1. Architecture du pipeline ETL

Le pipeline, orchestré par le script Python `etl_pipeline.py` (situé dans `backend/etl/`), coordonne l'extraction des données GPX, l'identification de la ville associée, la récupération d'informations complémentaires (météo, spots P4N, POI OSM, patrimoine Wikidata), la transformation de l'ensemble des données, et enfin leur chargement dans les bases de données MySQL et MongoDB. [🔗 Diagramme de flux ETL Pipeline](#)

Point de lancement : Automatique lors import GPX via frontend, ou manuel : `docker compose exec backend python3 -m etl.etl_pipeline.`

Initialisation dépendances : Conteneurisation Docker, Dockerfile.backend installe bibliothèques depuis `requirements.txt` (requests, selenium, gpxpy, pymongo, mysql-connector-python, pandas, geopy).

Gestion erreurs : Logging structuré `utils/logger_util.py` (fichiers rotatifs + console), try-catch permettant continuation si échec partiel, fallback 3 serveurs Overpass, validation cohérence avant chargement, fin traitement et sauvegarde résultats. [🔗 Exemples de logs](#)

2.2. Extraction des sources

Fichiers locaux - GPX [🔗 Diagramme de flux ETL des fichiers GPX](#)

Le module `gpx.py` effectue la lecture des fichiers GPX à partir du répertoire `data/`. Il sélectionne le premier fichier `.gpx` disponible et procède immédiatement au parsing XML pour extraire les données de base : les points de trace (latitude, longitude, élévation), les waypoints et les métadonnées.

API REST - Open-Meteo [🔗 Diagramme de flux ETL de gestion des données météo](#)

Le module `api_meteo.py` commence par géocoder la ville (via Nominatim) afin d'obtenir ses coordonnées. Il soumet ensuite une requête GET à l'API `api.open-meteo.com/v1/forecast`, en spécifiant ces coordonnées, les variables météorologiques quotidiennes désirées, et une période de prévision de 7 jours. Le module procède enfin au parsing du JSON de la réponse pour générer des tuples SQL et enregistrer des logs horodatés de l'opération.

Web scraping - Park4Night [🔗 Diagramme de flux ETL des données Park For Night](#)

Le module `scraper_p4n.py` utilise l'outil Selenium avec Chromium en mode sans tête (*headless*) pour automatiser la collecte d'informations sur Park4Night : il gère d'abord le bandeau de cookies, puis navigue vers une URL de recherche construite avec les coordonnées géographiques. Le script attend jusqu'à 40 secondes que la liste des résultats soit complètement chargée (vérifiant la présence de l'élément `ul class="listmap-list"`), avant d'extraire les liens vers les fiches détaillées des spots. Il procède ensuite à la collecte des données (nom, description, type, coordonnées, note, services) uniquement si le spot n'est pas déjà en base de données, et consolide l'ensemble des informations dans un DataFrame pandas.

Module : `scraper_p4n.py` utilise Selenium/Chromium en mode headless

Url ville : <https://park4night.com/fr/search?lat=48.34151&lng=-4.71924&z=15>

Url spot : <https://park4night.com/fr/place/225996>

API REST - Overpass API [🔗 Diagramme de flux ETL des données Open Street Map](#)

Le module `api_osm.py` commence par géocoder la ville pour définir une zone de recherche (bounding box) d'un rayon de 5 km. Il interroge ensuite l'API Overpass en utilisant le langage spécifique Overpass QL pour récupérer les POI variés (attractions, eau, toilettes, restaurants, supermarchés, carburant) via des requêtes comme `nwr["tourism"]="attraction"`. Le script est configuré pour basculer sur trois serveurs Overpass différents en cas d'échec (avec un délai d'attente de 90 secondes). Si les requêtes échouent malgré cela, un processus d'`etl_rattrapage` a été mis en place pour relancer l'extraction sur les villes concernées. La réponse de l'API est reçue au format JSON, contenant l'ID OSM, les coordonnées et les tags associés à chaque POI.

API REST - Wikidata [🔗 Diagramme de flux ETL des données Wikidata](#)

Le module `api_wikidata.py` interroge la base de données Wikidata en utilisant le langage SPARQL (via le service `query.wikidata.org/sparql`) pour récupérer les Points d'Intérêt situés dans un rayon de 5 km autour

de la ville. Les requêtes ciblent des identifiants Wikidata spécifiques (comme wd:Q570116 pour un Phare ou wd:Q23397 pour une Plage). La réponse est fournie au format JSON et contient l'ID QID Wikidata, les coordonnées, l'URL de l'image et le type de POI correspondant.

3. AGRÉGATION ET NETTOYAGE

3.1. Agrégation OSM

Le module `transform_osm.py` convertit le JSON de l'API Overpass en un DataFrame pandas. Il procède au nettoyage en supprimant les données corrompues (éléments sans tags ou coordonnées) et en normalisant les noms. L'étape clé est le mapping des POI : les données sont homogénéisées par l'assignation à des catégories métier (restaurant, hébergement, services, patrimoine, nature, autre). Le processus d'agrégation des données Wikidata suit ce même concept de transformation.

3.2. Agrégation Park4Night

Le module `transform_p4n.py` prend en charge le DataFrame pandas résultant du scraping de Park4Night. Il commence par le parsing des coordonnées (ex: "(48.3714, -4.7587)") pour les diviser en colonnes de latitude et longitude distinctes. Ensuite, il effectue un split de la chaîne de services pour mapper les types de services à un dictionnaire commun, assurant ainsi l'homogénéisation avec les données d'OSM et de Wikidata. Enfin, il valide les notes : il les convertit en numérique, remplace les erreurs de conversion ou les notes supérieures à 5 (ou absentes) par la valeur NaN.

3.3. Agrégation GPX

La phase de transformation calcule les indicateurs clés : la distance (utilisant la formule de Haversine par la méthode `length()`), le dénivelé positif cumulé, et la durée estimée selon Naismith (basée sur une vitesse de 4 km/h en plat et l'ajout d'une heure pour 600 mètres de montée). Enfin, il réalise un géocodage inverse pour identifier la ville du point de départ, information essentielle transmise aux autres processus ETL.

4. BASE DE DONNÉES

4.1. Base de données MySQL [MCD BDD MySQL, données structurées](#)

La structure des tables MySQL du projet Randovango est organisée autour des entités principales suivantes, elle stock toutes les données nécessaires à la planification d'une randonnée :

- **cities** : Contient les villes (nom, coordonnées, département, région, pays).
- **hikes** : Randonnées, liées à une ville (`city_id`), avec infos GPX, distance, dénivelé, source, etc.
- **sources** : Référence des sources de données (ex : auteur GPX).
- **spots** : Points d'intérêt (POI) géolocalisés, liés à une ville ou une randonnée.
- **poi** : Services ou POI spécifiques, avec type, coordonnées, etc.
- **weather** : Prévisions météo, liées à une ville (`city_id`) et une date.
- **trip_plans** : Plans de voyage créés par les utilisateurs, liés à une ville et à un utilisateur ou un token invité.
- **trip_days** : Jours associés à un plan, chaque jour pouvant référencer une randonnée (`hike_id`) et un spot (`spot_id`)
- **trip_days_pois** : sur un jour d'itinéraire l'utilisateur peut choisir plusieurs POI

Exemples de relations:

- `hikes.city_id` → `cities.id` (une randonnée appartient à une ville)
- `trip_plans.user_id` → `users.id` (ou `user_token` pour invités)
- `trip_days.trip_plan_id` → `trip_plans.id`

Cela forme un schéma relationnel centré sur la ville, avec des liens forts entre randonnées, plans, météo et POI. Les relations sont assurées par des clés étrangères, garantissant la cohérence des données. [Schéma de la base de données MySQL](#)

4.2. Base de données SQLite [MCD de la BDD SQLITE, authentification](#)

Le module d'authentification de Randovango repose sur une base SQLite légère, dédiée à la gestion des utilisateurs, des rôles et à l'audit des accès API.

- **users**: stocke les comptes utilisateurs (identifiant, nom d'utilisateur, mot de passe hashé, etc.).
- **roles** : liste des rôles disponibles (admin, contributeur et user).
- **user_role** : table d'association permettant d'attribuer plusieurs rôles à un utilisateur (clé composée user_id, role_id).
- **auth_log** : journalisation des connexions et actions pour l'audit de la sécurité.

4.3. Base de données Mongo

MongoDB est utilisé pour stocker les traces GPX brutes. Ce choix permet de conserver l'intégralité des fichiers d'origine, de faciliter les recherches géospatiales et d'assurer la flexibilité sur les formats de données importés. A terme le front pourra proposer ces fichiers directement aux utilisateurs pour qu'ils les exploitent lors de leur randonnée.

4.4. Requêtes SQL et optimisations [Exemples de requêtes](#)

Extraction villes enrichie :

Le module step1.py dans backend/api/routers/ implémente les requêtes SQL pour la sélection des villes. La requête principale extrait l'identifiant, le nom, le département, la région, le pays, la latitude et la longitude depuis la table cities, triés par ordre alphabétique de nom.

Cette requête de base est ensuite enrichie dynamiquement avec trois comptages distincts effectués pour chaque ville :

- récupère le nombre de randonnées vérifiées dans un rayon de 5 kilomètres. Le filtrage sur « verifie » garantit que seules les randonnées validées par les modérateurs soient présentées.
- récupère le nombre de spots van vérifiées dans un rayon de 5 kilomètres
- récupère l'ensemble des points d'intérêt toujours dans un rayon de 5 kilomètres

PS : A terme le filtrage sur 5 km, pourra être un paramètre côté front.

Extraction météo :

Le module load_meteo.py gère l'insertion des prévisions météorologiques avec une logique anti-doublons. Avant chaque insertion, une requête de vérification contrôle l'existence d'une prévision pour le couple (city_id, date). Si la requête retourne un résultat, l'insertion est ignorée.

La récupération des prévisions pour affichage frontend utilise la requête : `SELECT * FROM weather WHERE city_id = %s ORDER BY date ASC`. Le tri chronologique est essentiel pour l'affichage sous forme de timeline météo sur 7 jours.

PS : Un etl_meteo spécifique a été créé pour rafraichir les données côté front pour toujours présenter 7 jours de données par rapport à la date du jour.

5. FASTAPI (API REST) [Documentation Swagger UI](#)

FastAPI est utilisé comme serveur principal pour exposer l'API REST du projet : il gère les routes pour la gestion des villes, randonnées, plans, utilisateurs, l'authentification JWT, et orchestre les appels aux modules ETL et aux bases de données. Il assure la rapidité, la sécurité et la documentation interactive de l'API Randovango.

Exemples de routes FastAPI

- **GET /cities** : liste les villes disponibles avec statistiques et météo.
- **POST /create_plan** : crée un nouveau plan de voyage (trip plan) pour un utilisateur ou un invité.
- **GET /hikes/{city_id}** : retourne les randonnées vérifiées pour une ville donnée.
- **POST /auth/login** : authentifie un utilisateur et retourne un token JWT.
- **DELETE /gpx/{gpx_id}** : supprime une trace GPX sur MySQL et stockage MongoDB pour les traces non validées par admin.

Ces routes illustrent la diversité des fonctionnalités exposées par l'API Randovango (gestion des données, planification, sécurité, personnalisation).

6. CONCLUSION

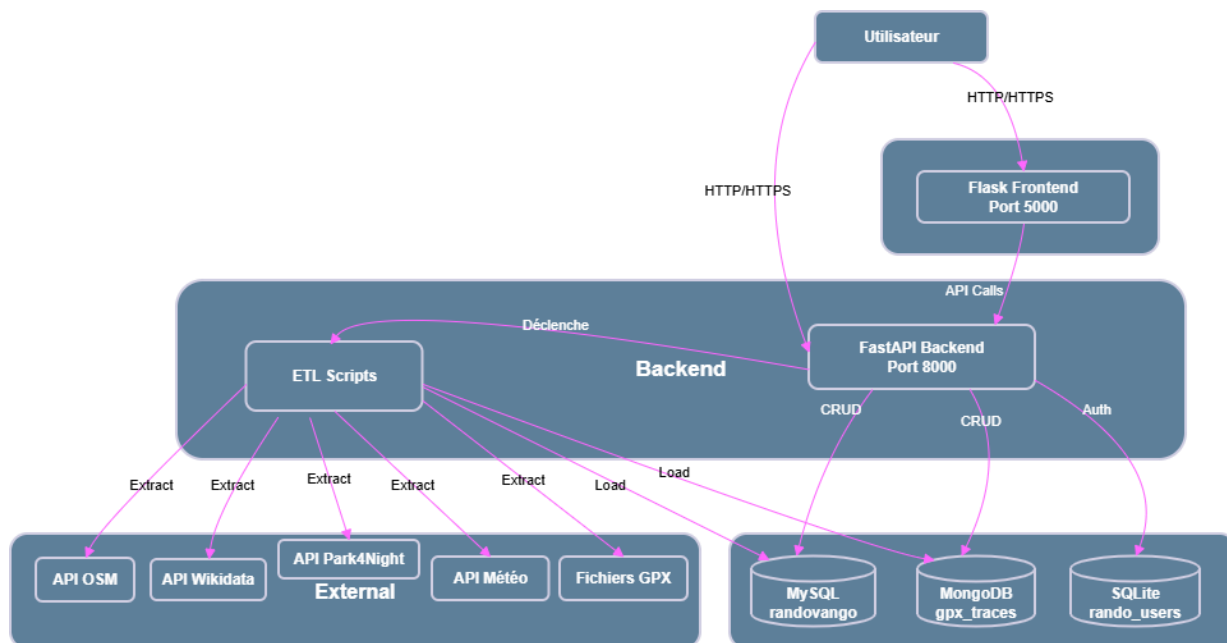
Le projet RandoVanGo pose les fondations d'une plateforme évolutive grâce à son architecture modulaire et ses flux de données automatisés. Plusieurs axes d'amélioration sont identifiés pour enrichir l'expérience utilisateur et renforcer la valeur ajoutée. L'interface utilisateur finale, accessible via le frontend Flask, concrétise l'ensemble de cette chaîne de traitement en proposant aux randonneurs en van une expérience complète de planification de séjours itinérants. [Vues du front](#)

L'intégration de modules d'intelligence artificielle constitue une évolution naturelle du projet : recommandations personnalisées d'itinéraires basées sur l'historique utilisateur et les préférences déclarées, analyse prédictive des conditions météorologiques combinant données historiques et tendances saisonnières, génération automatique de plannings optimisés selon le profil physique et les contraintes temporelles. Ces fonctionnalités s'appuieraient sur les données déjà collectées et stockées, minimisant l'impact technique de leur implémentation.

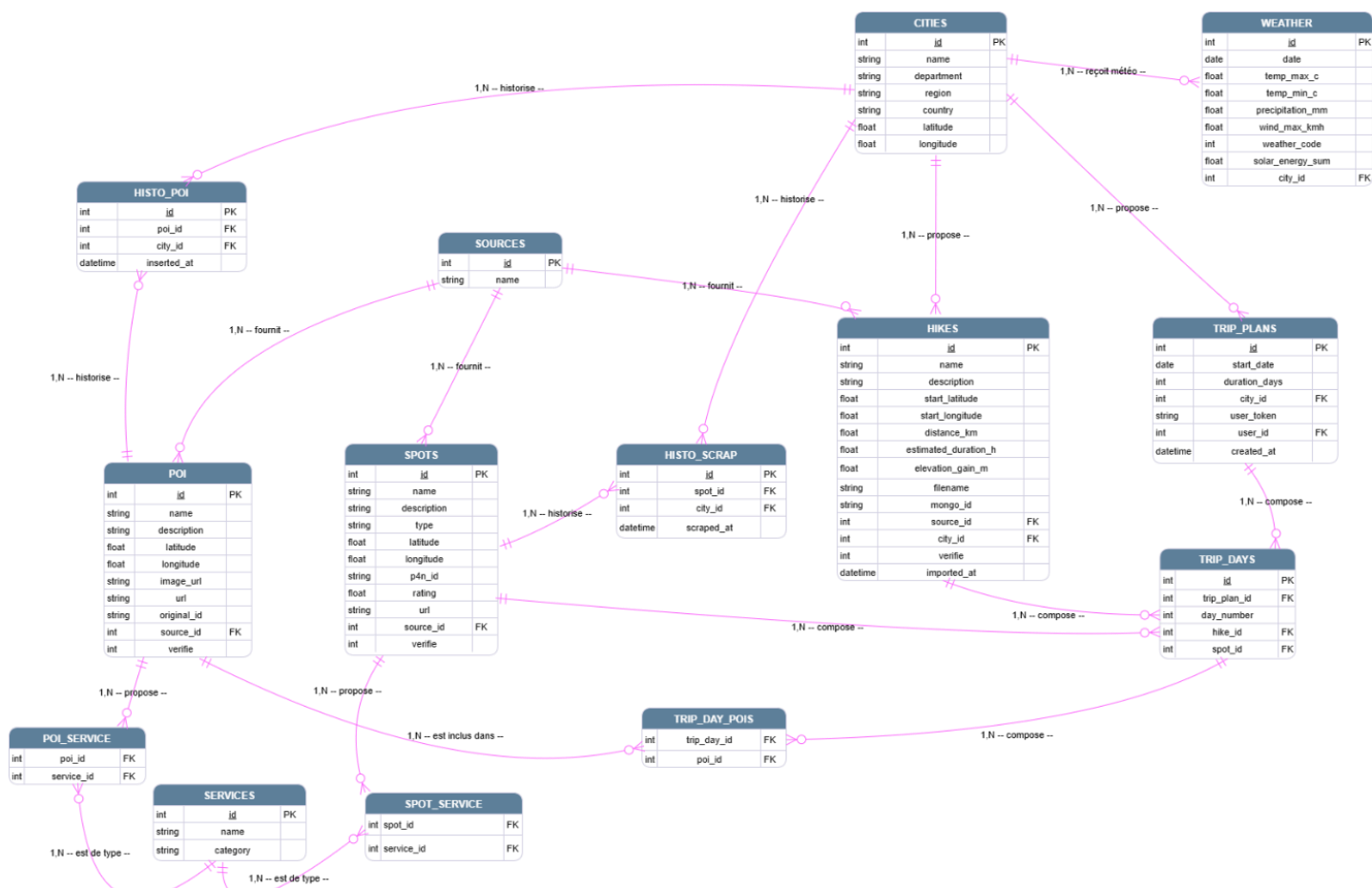
La conception modulaire du pipeline ETL facilite l'adaptation à d'autres pratiques itinérantes (bikepacking, trekking sac à dos). L'ajout de nouvelles sources de données ou de critères de filtrage spécifiques (difficulté technique, accessibilité PMR, présence de refuges) ne nécessiterait que l'ajout de modules ETL supplémentaires sans remise en cause de l'architecture existante.

7. ANNEXES

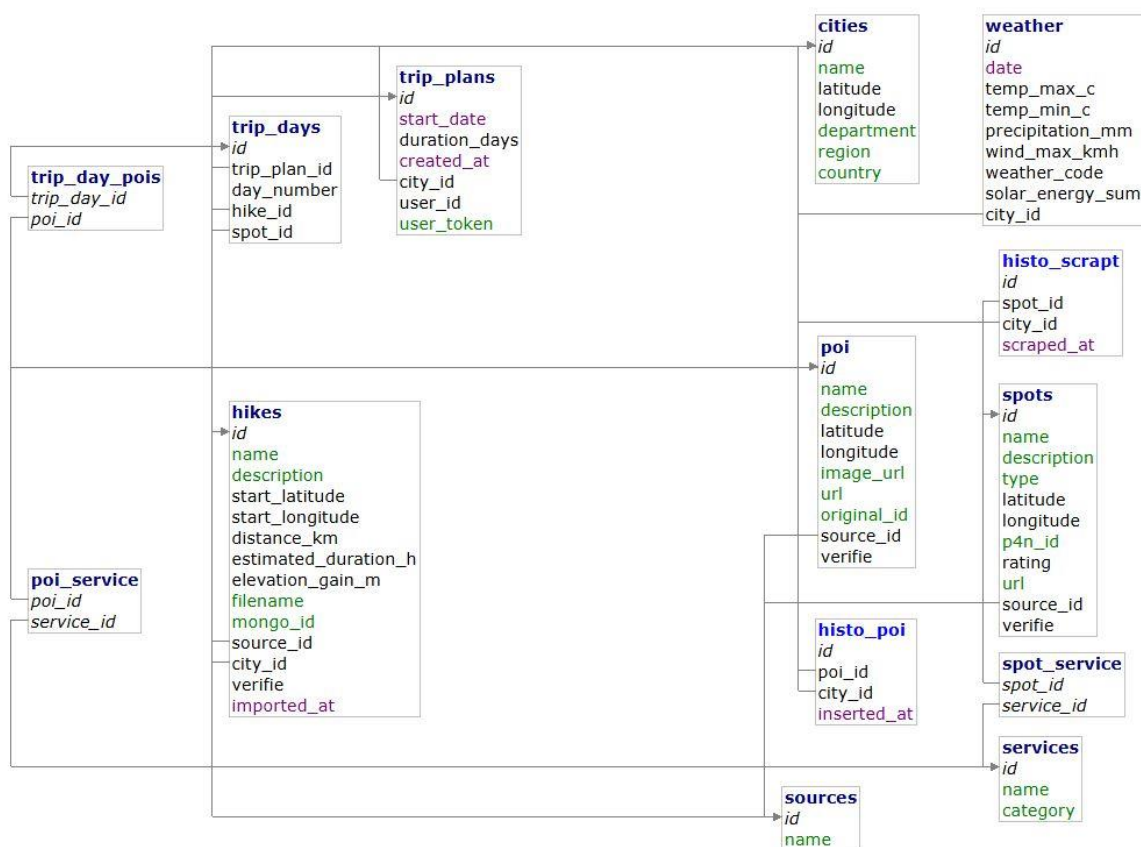
7.1. Architecture technique globale [Retour](#)



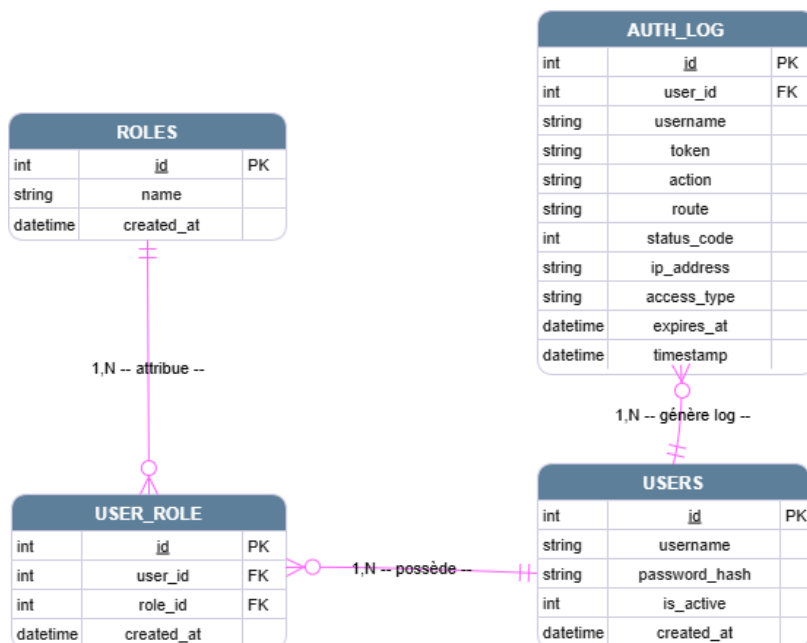
7.2. MCD BDD MySQL, données structurées nécessaire à la planification [Retour](#)



7.3. Schéma de la base de données MySQL [Retour](#)



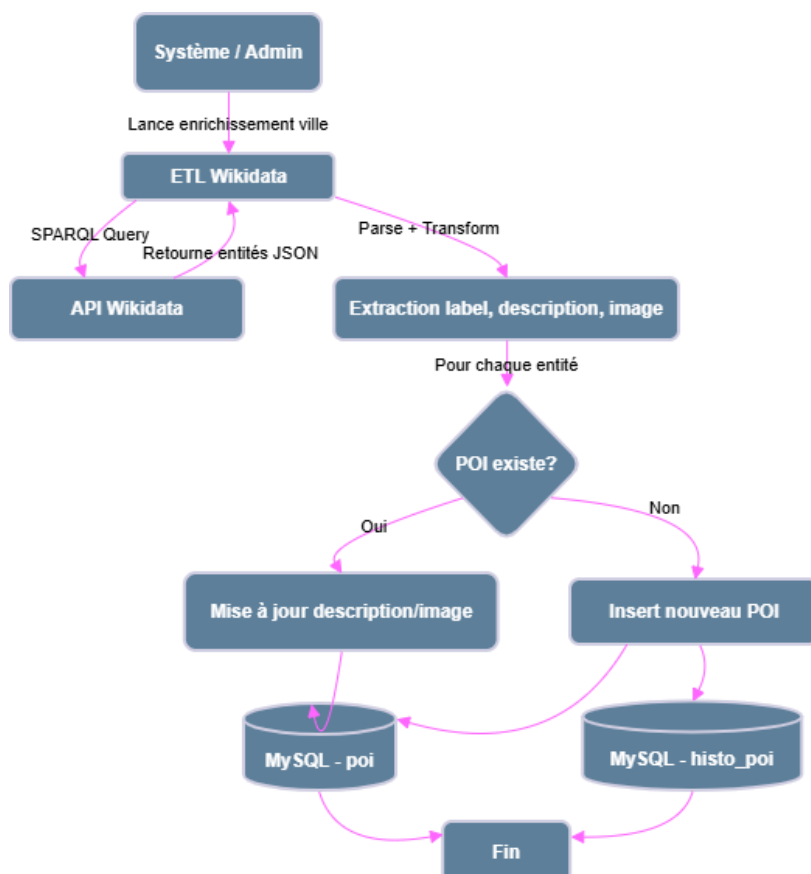
7.4. MCD de la BDD SQLITE, authentification [Retour](#)



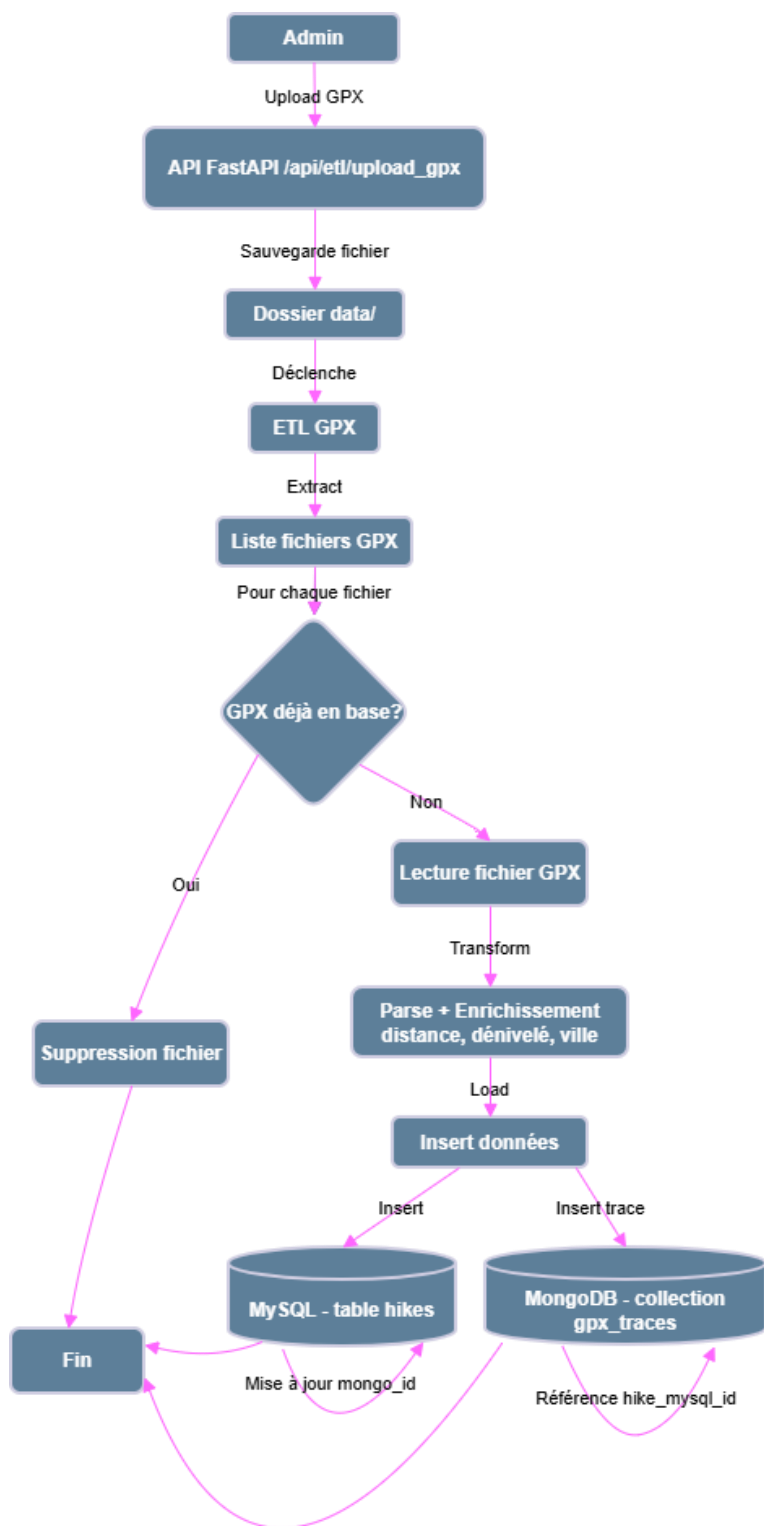
7.5. Diagramme de flux ETL Pipeline [Retour](#)



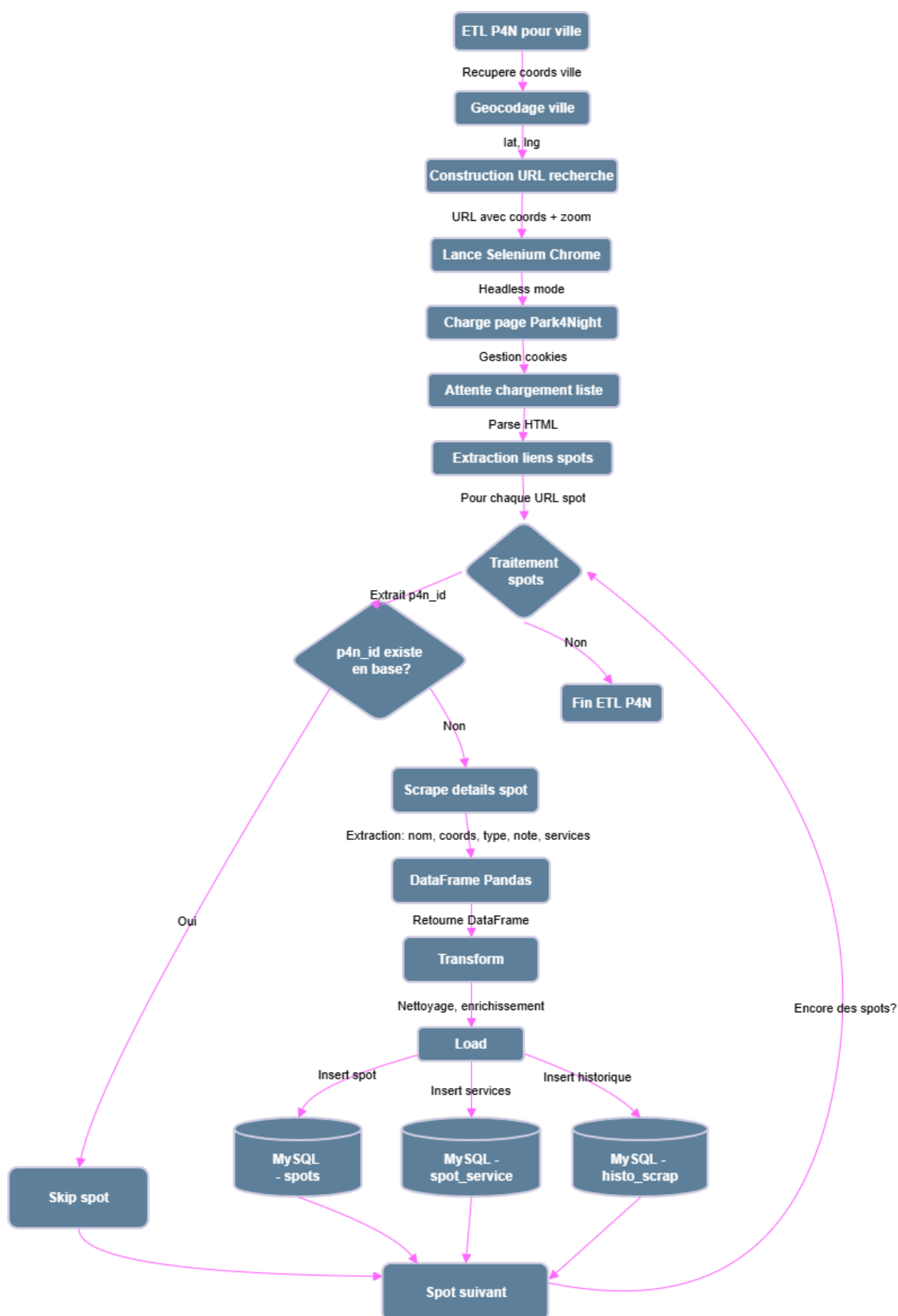
7.6. Diagramme de flux ETL des données Wikidata [Retour](#)



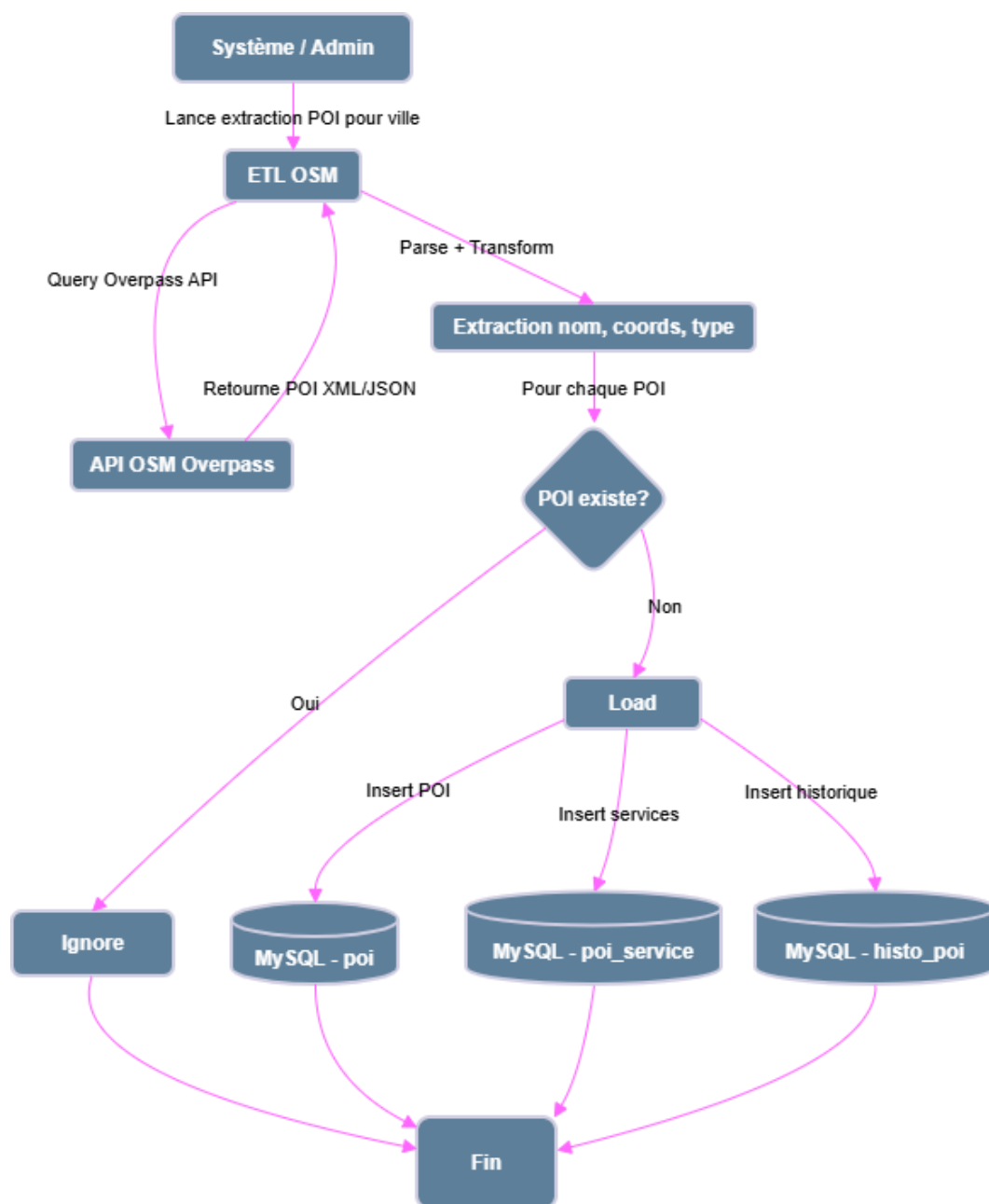
7.7. Diagramme de flux ETL des fichiers GPX [Retour](#)

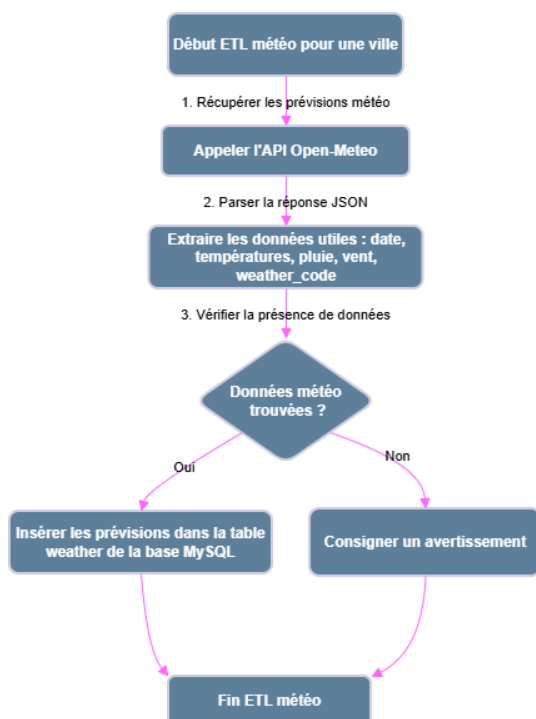


7.8. Diagramme de flux ETL des données Park For Night [Retour](#)



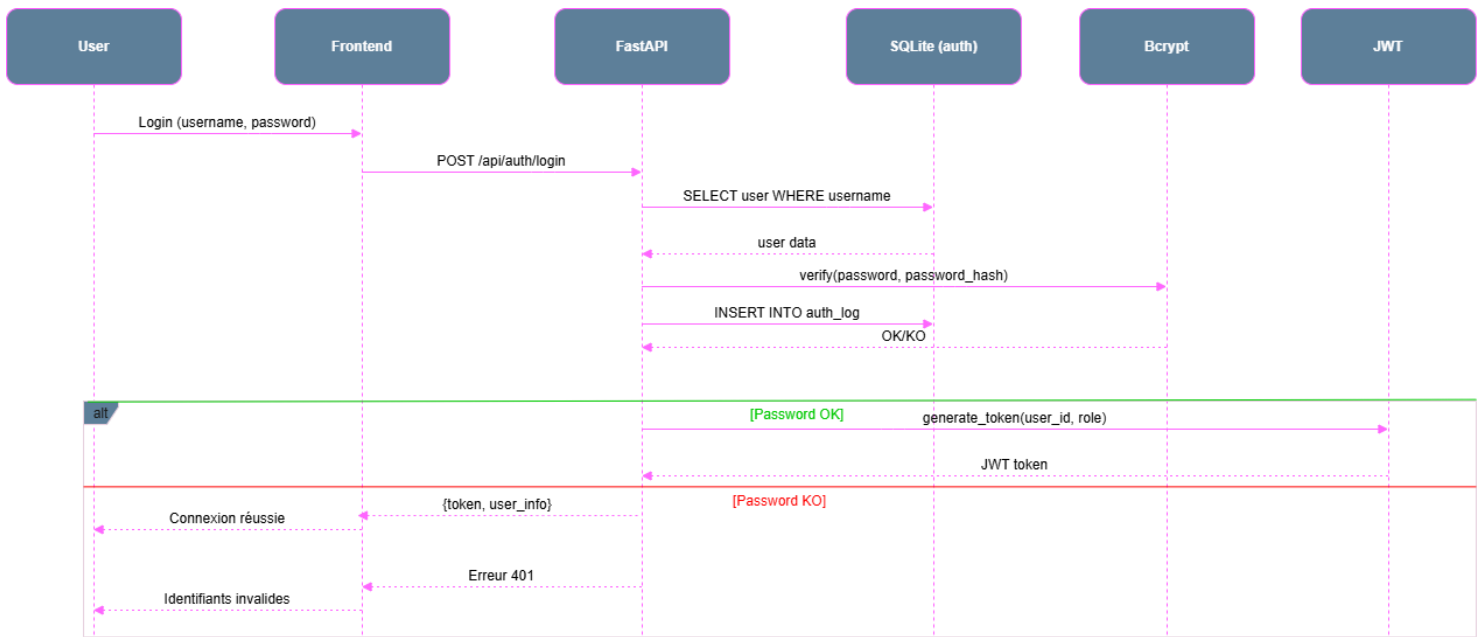
7.9. Diagramme de flux ETL des données Open Street Map [Retour](#)



7.10. Diagramme de flux ETL de gestion des données météo [Retour](#)7.11. Exemples de requêtes [Retour](#)

```
docs > requetes.sql
1  -- Annexe - Requêtes SQL réelles utilisées dans le projet Randovango (avec références)
2
3  -- 1. Sélection du nombre de randonnées vérifiées dans un rayon autour d'une ville
4  -- Fichier : backend/api/routers/step1.py
5  SELECT COUNT(*) as count FROM hikes
6  WHERE verifie = 1 AND start_latitude BETWEEN :min_lat AND :max_lat AND start_longitude BETWEEN :min_lon AND :max_lon;
7
8  -- 2. Vérification de l'existence de prévisions météo à J+6
9  -- Fichier : backend/api/routers/step1.py
10 SELECT COUNT(*) as count FROM weather WHERE date >= :date_j6;
11
12 -- 3. Sélection des villes (pour affichage)
13 -- Fichier : backend/api/routers/step1.py
14 SELECT id, name, department, region, country, latitude, longitude FROM cities ORDER BY name ASC;
15
16 -- 4. Sélection des prévisions météo pour une ville
17 -- Fichier : backend/api/routers/step1.py
18 SELECT * FROM weather WHERE city_id = :city_id ORDER BY date ASC;
19
20 -- 5. Création d'un nouveau plan de voyage (trip_plans)
21 -- Fichier : backend/api/routers/step1.py
22 INSERT INTO trip_plans (start_date, duration_days, city_id, user_token, user_id, created_at)
23 VALUES (:start_date, :duration_days, :city_id, :user_token, :user_id, NOW());
24
25 -- 6. Création des jours associés à un plan (trip_days)
26 -- Fichier : backend/api/routers/step1.py
27 INSERT INTO trip_days (trip_plan_id, day_number, hike_id, spot_id) VALUES (:trip_plan_id, :day_number, NULL, NULL);
28
29 -- 7. Mise à jour du champ mongo_id dans une randonnée (GPX)
30 -- Fichier : backend/etl/load/load_gpx.py
31 UPDATE hikes SET mongo_id = :id_mongo WHERE id = :gpxtrace_id;
32
33 -- 8. Sélection des rôles d'un utilisateur (jointure roles/user_role)
34 -- Fichier : backend/services/authentification.py
35 SELECT r.name FROM roles r JOIN user_role ur ON ur.role_id = r.id WHERE ur.user_id = :user_id;
36
37 -- 9. Sélection du nombre de scrapings pour une ville (jointure histo_scrap/cities)
38 -- Fichier : backend/etl/etl_pipeline.py
39 SELECT COUNT(*) FROM histo_scrap hs JOIN cities c ON hs.city_id = c.id WHERE c.name = :city_name;
```

7.12. Diagramme de séquence authentification [Retour](#)



7.13. Documentation Swagger UI [Retour](#)

RandoVanGo API 0.1.0 OpenAPI 3.0
/openapi.json
API du planificateurs

Authorize

Step 1 Cities

- GET /api/step1/cities Returns the list of cities with statistics and updated weather forecasts.
- POST /api/step1/create_plan Create a new trip plan with empty days.

Step 2 Hikes

- GET /api/step2/hikes Returns the hikes for a given city.
- PUT /api/step2/update_plan/{plan_id} Update an existing trip plan with new data.

Step 3 Spots

- GET /api/step3/spots Returns the spots for a given city and hike.
- PUT /api/step3/update_plan/{plan_id} Update an existing trip plan with new data.

Step 4 Services

- GET /api/step4/services Returns the services for a given city, hike, and spot.
- PUT /api/step4/update_plan/{plan_id} Update an existing trip plan with new data.

Result Plan

- GET /api/result/ Retrieve the trip plan details by plan ID.

Authentication

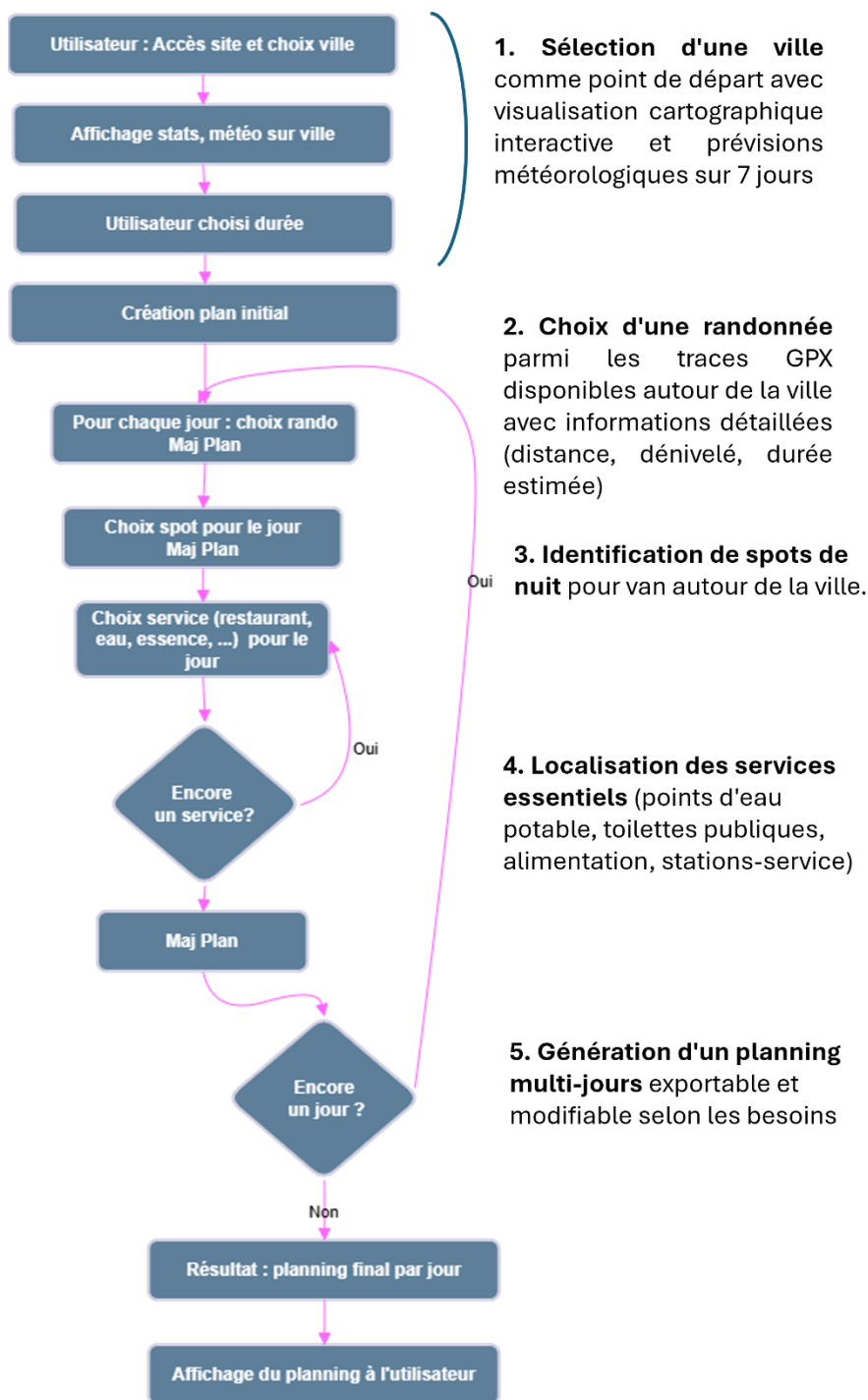
- POST /api/auth/login Login user and return JWT token.
- POST /api/auth/logout Logout user.

GPX Files

- POST /api/etl/upload_gpx Upload a GPX file and launch the ETL pipeline (Admin and Contributor only).
- DELETE /api/etl/delete_gpx/{filename} Delete a GPX file (Admin only).

Schemas

7.14. Gestion Planning Rando [Retour](#)



7.15. Arborescence projet [Retour](#)

```

randovango/
├── backend/
│   ├── main.py
│   ├── api/
│   │   ├── models/
│   │   ├── routers/
│   │   └── services/
│   ├── db_init/
│   │   ├── db_randovango.py
│   │   └── db_users.py
│   ├── etl/
│   │   ├── etl_meteo.py
│   │   ├── etl_pipeline.py
│   │   ├── etl_rattrapage.py
│   │   ├── extract/
│   │   │   ├── api_meteo.py
│   │   │   ├── api_osm.py
│   │   │   ├── api_wikidata.py
│   │   │   ├── gpx.py
│   │   │   └── scraper_p4n.py
│   │   ├── load/
│   │   │   ├── load_gpx.py
│   │   │   ├── load_meteo.py
│   │   │   ├── load_p4n.py
│   │   │   ├── load_poi.py
│   │   └── transform/
│   │       ├── transform_gpx.py
│   │       ├── transform_osm.py
│   │       ├── transform_p4n.py
│   │       └── transform_wikidata.py
│   ├── services/
│   │   ├── authentication.py
│   │   └── plan_service.py
│   └── utils/
│       ├── geo_utils.py
│       ├── logger_util.py
│       ├── meteo_utils.py
│       ├── mongo_utils.py
│       ├── mysql_utils.py
│       └── service_utils.py
├── data/
├── docs/
├── frontend/
│   ├── app.py
│   ├── static/
│   │   ├── images/
│   │   └── styles.css
│   └── templates/
│       ├── base.html
│       └── pages/
│           ├── results.html
│           ├── step1_city.html
│           ├── step2_hike.html
│           ├── step3_spot.html
│           └── step4_services.html
├── logs/
├── storage/
│   ├── mongodb/
│   ├── mysql/
│   └── rando_users.sqlite
├── requirements.txt
├── docker-compose.yml
├── Dockerfile.backend
├── Dockerfile.frontend
├── README.md
└── .env

```

7.16. Exemples de logs Retour

```
logs >  scraper_p4n.log
0531 2025-10-29 14:08:02,503 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/002 ---
6332 2025-10-29 14:08:03,203 - INFO - Extrait: (14170) Le Billot - D39
6333 2025-10-29 14:08:04,241 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/353250 ---
6334 2025-10-29 14:08:04,927 - INFO - Extrait: (61120) Vimoutiers - 10 Avenue du Général de Gaulle
6335 2025-10-29 14:08:05,967 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/2016 ---
6336 2025-10-29 14:08:07,275 - INFO - Extrait: (61120) Vimoutiers - Rue du Docteur Dentu
6337 2025-10-29 14:08:08,295 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/440818 ---
6338 2025-10-29 14:08:09,218 - INFO - Extrait: (61250) Saint-Nicolas-des-Bois - 81 L'Ermitage
6339 2025-10-29 14:08:10,264 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/451754 ---
6340 2025-10-29 14:08:13,007 - INFO - Extrait: (61250) Écouves - 3 Route du Salut
6341 2025-10-29 14:08:14,043 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/205015 ---
6342 2025-10-29 14:08:14,948 - INFO - Extrait: (61120) Camping Municipal La Campière - Vimoutiers
6343 2025-10-29 14:08:15,983 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/289282 ---
6344 2025-10-29 14:08:16,751 - INFO - Extrait: (14140) Val-de-Vie - D179A
6345 2025-10-29 14:08:17,772 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/78531 ---
6346 2025-10-29 14:08:18,624 - INFO - Extrait: (14140) Cave Cibricole Gautar
6347 2025-10-29 14:08:19,663 - INFO - --- Scraping de l'URL : https://park4night.com/fr/place/113959 ---

logs >  api_osm.log
26 2025-10-29 09:37:31,147 - INFO - Données OSM extraites pour Saint-Renan (302 POI)
27 2025-10-29 09:50:03,627 - INFO - Lancement de l'extraction OSM via Overpass pour 'Saint-Renan'...
28 2025-10-29 09:50:03,628 - INFO - Recherche des coordonnées pour 'Saint-Renan'...
29 2025-10-29 09:50:03,858 - INFO - Coordonnées trouvées : 48.4328451, -4.622982
30 2025-10-29 09:50:03,859 - INFO - Bounding box : {'south': 48.382845100000004, 'north': 48.4828451, 'west': -4.672982, 'east':
31 2025-10-29 09:50:03,859 - INFO - Tentative de connexion au serveur : https://overpass-api.de/api/interpreter
32 2025-10-29 09:50:05,986 - INFO - Lancement de l'extraction OSM via Overpass pour 'Portsall'...
33 2025-10-29 09:50:05,986 - INFO - Recherche des coordonnées pour 'Portsall'...
34 2025-10-29 09:50:06,694 - INFO - Coordonnées trouvées : 48.5578742, -4.6982378
35 2025-10-29 09:50:06,696 - INFO - Bounding box : {'south': 48.5078742, 'north': 48.6078742, 'west': -4.7482378, 'east': -4.6482
36 2025-10-29 09:50:06,698 - INFO - Tentative de connexion au serveur : https://overpass-api.de/api/interpreter
37 2025-10-29 09:50:10,824 - INFO - Nombre d'éléments trouvés : 302
38 2025-10-29 09:50:10,825 - INFO - Succès : 302 POI trouvés pour 'Saint-Renan'
39 2025-10-29 09:50:10,826 - INFO - Données OSM extraites pour Saint-Renan (302 POI)
40 2025-10-29 09:50:13,186 - WARNING - Erreur de connexion au serveur : https://overpass-api.de/api/interpreter : 504 Server Error:
41 2025-10-29 09:50:13,186 - INFO - Tentative de connexion au serveur : https://overpass.kumi.systems/api/interpreter
42 2025-10-29 09:51:04,122 - INFO - Nombre d'éléments trouvés : 307
43 2025-10-29 09:51:04,123 - INFO - Succès : 307 POI trouvés pour 'Portsall'
44 2025-10-29 09:51:04,123 - INFO - Données OSM extraites pour Portsall (307 POI)
45 2025-10-29 09:58:00,473 - INFO - Lancement de l'extraction OSM via Overpass pour 'Tal ar Groas'...
46 2025-10-29 09:58:00,473 - INFO - Recherche des coordonnées pour 'Tal ar Groas'...
47 2025-10-29 09:58:01,825 - INFO - Coordonnées trouvées : 48.2482858, -4.4182903
48 2025-10-29 09:58:01,826 - INFO - Bounding box : {'south': 48.1982858, 'north': 48.298285799999995, 'west': -4.4682903, 'east':
49 2025-10-29 09:58:01,827 - INFO - Tentative de connexion au serveur : https://overpass-api.de/api/interpreter
50 2025-10-29 09:58:03,595 - INFO - Nombre d'éléments trouvés : 158
51 2025-10-29 09:58:03,596 - INFO - Succès : 158 POI trouvés pour 'Tal ar Groas'
52 2025-10-29 09:58:03,596 - INFO - Données OSM extraites pour Tal ar Groas (158 POI)
```

7.17. Vues du front [Retour](#)

Planificateur RandoVanGo

Choisissez votre destination pour commencer l'aventure

Pour insérer de nouvelles randonnées !

Connexion Contributeur

1 Ville

2 Randonnée

3 Nuit

4 Services

Résultat

Planifiez votre séjour RandoVanGo

Choisissez votre destination et la durée de votre aventure

Lila

Finistère - France

2 randonnées 36 spots nuit 58 points d'intérêt

Visualisation sur la carte

Météo à Lila - Prochains jours

Consultez les prévisions locales pour choisir la durée idéale de votre séjour

Aujourd'hui 14° - 12° 7.8 mm 26 km/h Équipements conseillés	Jeudi 30 oct 15° - 9° 3.1 mm 43 km/h Équipements conseillés	Vendredi 31 oct 16° - 14° 1.7 mm 44 km/h Équipements conseillés	Samedi 1 nov 14° - 12° 6.0 mm 45 km/h Équipements conseillés	Dimanche 2 nov 15° - 12° 8.3 mm 48 km/h Équipements conseillés	Lundi 3 nov 16° - 15° 0.2 mm 43 km/h Conditions correctes	Mardi 4 nov 16° - 14° 35 km/h Conditions correctes	Mercredi 5 nov 16° - 12° 2.9 mm 42 km/h Équipements conseillés
--	--	--	---	---	--	--	---

Combien de jours voulez-vous rester ?

Consultez la météo ci-dessus pour choisir la durée idéale de votre séjour

1
jour
Escapade express

2
jours
Week-end parfait

3
jours
Long week-end

7
jours
Semaine complète

?
jours
Durée personnalisée

Planifier ces 2 jours →